Testing and Validation

Blue Harvest · Mortgage Pricing Exceptions

Purpose: explain the selected test levels, demonstrate coverage of important business rules and record actual local test results. Existing suites were inspected and executed for this document. No application or test code was changed.

1. Selected Test Levels

1.1 Service-level unit tests

PricingExceptions.UnitTests calls services directly using isolated EF Core InMemory stores. These fast checks focus on business transitions, roles, revisions, history, retries and invalid inputs. They also cover development-user seeding. They do not prove PostgreSQL constraints or transaction behavior.

1.2 HTTP integration tests

PricingExceptions.IntegrationTests uses the ASP.NET Core test host and an isolated InMemory database. It verifies routing, request binding, validation, development authentication, authorization, DTO responses, HTTP status codes, Location headers and Swagger. This proves the HTTP pipeline, not the deployed Nginx proxy or relational guarantees.

1.3 PostgreSQL integration tests

PricingExceptions.PostgreSqlTests starts a disposable PostgreSQL container and uses separate migrated databases. It checks foreign keys, unique revision/decision constraints, transaction rollback, persistence across contexts, stale updates and competing operations. Docker is required. These tests do not establish correctness across multiple deployed API instances.

1.4 Browser end-to-end tests

Playwright runs FrontEnd/tests/workspace.spec.ts and theme.spec.ts against isolated API and Vite servers. It checks user interactions, validation, role switching, decision attribution, request history, retries, responsive layouts and themes. On Windows the configured browser is Microsoft Edge. The isolated API uses InMemory, so the browser suite complements rather than replaces PostgreSQL tests.

2. Business Coverage

Backend test paths below are relative to Backend/BlueHarvest.PricingExceptions/tests. Tests with parameterized inputs execute multiple cases.

2.1 Request creation and validation

PricingExceptionServiceTests.ManagerCanCreateRequestWithApplicationDiscountReasonAndReviewer checks saved application, terms and reviewer. InvalidDiscountOrReasonDoesNotCreateRequest and InvalidReviewerAssignmentDoesNotCreateRequest verify rejected input leaves no request or completed retry record. HTTP tests additionally reject invalid application IDs, missing reasons and missing idempotency keys.

2. Business Coverage (continued)

2.2 Approval, decline and decision history

ApprovingRequestSetsStatusToApproved and DecliningRequestSetsStatusToDeclined check service transitions. AssignedReviewerCanApprovePendingRequest and AssignedReviewerCanDeclineRequestWithReason verify HTTP outcomes and attribution. RequestHistoryPreservesPreviousRevisionsAndDecisions and RequestHistoryPreservesRevisionsDecisionsAndBusinessActions check retained terms and decisions. History tests also preserve name snapshots and reject changes or deletion of submitted records.

2.3 Duplicate prevention and retries

DuplicateRequestIsRejected checks the one-request-per-application rule. RetryingRequestDoesNotCreateDuplicate checks replay. HTTP tests include RetryingRequestCreationReturnsOriginalResponse, RetryingReviewDoesNotCreateDuplicateDecision, RetryingRevisionDoesNotCreateDuplicateRevision and ReusingIdempotencyKeyWithDifferentDataIsRejected. PostgreSQL tests cover duplicate applications across independent contexts and concurrent creation retries.

2.4 Approved-discount retrieval and revisions

ApprovedDiscountIsAvailableToMortgageProcess verifies approved terms are accessible to the mortgage process. UnapprovedDiscountIsNotAvailableToMortgageProcess and HTTP UnapprovedRequestDoesNotExposeDiscount cover Pending and Declined outcomes. RevisingApprovedRequestRequiresNewApproval and HTTP SubmittingRevisionRequiresNewApproval check that a revision withdraws effective approval until a new decision. Earlier approvals remain in history and are not a fallback.

2.5 Access rules, concurrency and transactional safety

Tests reject an unassigned reviewer, self-review, another manager's revision and stale version/revision values. PostgreSQL tests include DatabaseAllowsOnlyOneDecisionPerRevision, DatabaseRejectsUpdateFromAnOutdatedContext, ConcurrentApprovalAndDeclinePersistOneDecision and ConcurrentReviewAndRevisionPersistOneChange. FailedSaveRollsBackRequestDecisionHistoryAndRetryRecord injects a foreign-key failure and checks from a fresh context that no partial changes survived.

3. Exact Test Commands

Prerequisites: .NET 10 SDK, Node.js 24 with npm, Docker Desktop using Linux containers and Microsoft Edge on Windows. Start Docker Desktop. Use separate terminals and run backend and browser commands sequentially to avoid overlapping API builds.

3.1 Full backend suite

Open PowerShell at the repository root:

```
cd Backend/BlueHarvest.PricingExceptions
dotnet test
```

dotnet test restores and builds the projects before running all three backend suites. The database fixture creates an isolated PostgreSQL container, separate from the main Compose database.

3. Exact Test Commands (continued)

3.2 Individual backend levels

Run from Backend/BlueHarvest.PricingExceptions:

```
dotnet test tests/PricingExceptions.UnitTests/PricingExceptions.UnitTests.csproj
dotnet test tests/PricingExceptions.IntegrationTests/PricingExceptions.IntegrationTests.csproj
dotnet test tests/PricingExceptions.PostgreSqlTests/PricingExceptions.PostgreSqlTests.csproj
```

These individual commands are provided for targeted reruns; the full solution command, rather than separate reruns, produced the backend evidence in Section 4.

3.3 Frontend browser tests

Open a new PowerShell terminal at the repository root:

```
cd FrontEnd
npm.cmd ci
npm.cmd run test:e2e
```

Dependency installation is a setup step. Existing dependencies were used for the observed run. Leave E2E_BASE_URL unset. The runner starts isolated servers on ports 5174 and 5270; release those ports before testing. Use npm instead of npm.cmd outside Windows and install Playwright Chromium where Edge is not configured.

3.4 Additional frontend checks and live API smoke test

From FrontEnd, use the following build and lint checks when reviewing a change:

```
npm.cmd run build
npm.cmd run lint
```

For a live PostgreSQL-backed deployment check, run from the repository root:

```
docker compose up -d --build
powershell.exe -NoProfile -ExecutionPolicy Bypass `
-File Backend/BlueHarvest.PricingExceptions/tests/Invoke-ApiSmokeTests.ps1 `
-BaseUrl http://localhost:5173
```

Wait for API startup before the smoke command. It creates persistent demo records. Build/lint and the live smoke commands are repository-verified instructions; they were not rerun during this documentation task. Previous runs are not counted as new evidence here.

4. Observed Passing Results

Executed locally on 7 November 2026 against the current working tree. Results are observed command output, not remote CI evidence or a clean-checkout certification.

```
dotnet test: exit code 0
Unit: Passed 56, Failed 0, Skipped 0
HTTP: Passed 118, Failed 0, Skipped 0
PostgreSQL: Passed 10, Failed 0, Skipped 0
Backend total: 184 passed

npm.cmd run test:e2e: final exit code 0
20 passed (47.1s)
```

4. Observed Results and Limitations (continued)

4.1 Initial failure and successful rerun

The first browser attempt failed before test execution because the configured API web server could not start; its build reported failure while backend tests were also building. The available output does not prove the root cause. After the backend suite finished, the same browser command was rerun separately and all 20 tests passed. No source change was made to obtain that result.

4.2 What passing tests demonstrate

The observed runs support the covered local creation, validation, decision, history, duplicate-prevention and discount-retrieval behavior. The PostgreSQL suite adds relational constraint and rollback evidence. The browser rerun demonstrates the configured API and frontend can start and complete the tested user scenarios in the local test environment.

4.3 What they do not demonstrate

These results do not establish production authentication, real mortgage-system integration, security hardening, load capacity, multi-instance race safety or complete recovery readiness. The current service-level duplicate checks should not be confused with database uniqueness guarantees. No percentage code coverage is claimed; no coverage measurement was taken for this task.

The main Compose application was not started as part of these isolated suite runs. A live smoke test remains a separate deployment check. Reproduce the commands against the intended submission commit and record any environment differences before relying on the result.

4.4 Source references

Unit source: tests/PricingExceptions.UnitTests/PricingExceptionServiceTests.cs and DevelopmentDataSeederTests.cs. HTTP source: tests/PricingExceptions.IntegrationTests/HttpWorkflowTests.cs. PostgreSQL source: tests/PricingExceptions.PostgreSqlTests/PostgreSqlWorkflowTests.cs and PostgreSqlFixture.cs. Backend paths are relative to Backend/BlueHarvest.PricingExceptions.

Browser source: FrontEnd/tests/workspace.spec.ts, FrontEnd/tests/theme.spec.ts and FrontEnd/playwright.config.ts. Scripts: FrontEnd/package.json and Backend/BlueHarvest.PricingExceptions/tests/Invoke-ApiSmokeTests.ps1. A concise observed-results record accompanies this document as Testing_and_Validation_Results.txt.